

# NED University of Engineering & Technology

Department of Computer Science & Information Technology

---

Course: Object Oriented Programming — Code: CT-260

---

## GalacticTrade

Space Merchant Simulator

*A Console-Based OOP Project in C++*

---

### Project Report

| Name               | Roll No. |
|--------------------|----------|
| Faarid Amir Shaikh | CT-25276 |
| Amna Ali Kaleem    | CT-25264 |
| Zainab Nadeem      | CT-25265 |

Submitted to the Department of Computer Science & IT

NED University of Engineering & Technology, Karachi

May 10, 2026

# Contents

|     |                                         |    |
|-----|-----------------------------------------|----|
| 1   | Problem Statement                       | 1  |
| 2   | OOP Principles Identification           | 1  |
| 2.1 | Encapsulation . . . . .                 | 1  |
| 2.2 | Inheritance . . . . .                   | 2  |
| 2.3 | Polymorphism . . . . .                  | 2  |
| 2.4 | Abstraction . . . . .                   | 3  |
| 2.5 | Static Members . . . . .                | 3  |
| 2.6 | Composition and Aggregation . . . . .   | 3  |
| 2.7 | Templates . . . . .                     | 3  |
| 3   | System Design & UML                     | 4  |
| 3.1 | UML Class Diagram . . . . .             | 4  |
| 3.2 | Class Responsibilities . . . . .        | 5  |
| 3.3 | Key Relationships . . . . .             | 6  |
| 4   | Solution Definition                     | 6  |
| 5   | Design Pattern Application              | 11 |
| 5.1 | Pattern Used: Singleton . . . . .       | 11 |
| 5.2 | Why Singleton for GameEngine? . . . . . | 11 |
| 5.3 | Implementation . . . . .                | 11 |
| 5.4 | How It Improves the Design . . . . .    | 12 |
| 5.5 | Usage in main.cpp . . . . .             | 12 |
| 6   | Limitations & Future Work               | 12 |
| 6.1 | Current Limitations . . . . .           | 12 |
| 6.2 | Potential Future Enhancements . . . . . | 13 |
| 7   | Team Contribution Table                 | 13 |
|     | References                              | 15 |

## 1. Problem Statement

---

Modern commodity trading systems rely on complex networks of supply, demand, pricing, and logistics. Managing these systems effectively — tracking resources, reacting to market events, and making real-time decisions — presents a meaningful computing challenge that naturally maps to object-oriented principles.

**GalacticTrade** is a console-based C++ simulation that models a real-world commodity trading and logistics system through a space exploration theme. Players take on the role of an interstellar merchant, travelling between planets, buying and selling resources at fluctuating market prices, managing fuel and cargo capacity, and responding to unpredictable economic events such as market crashes, resource booms, and pirate attacks.

Although the setting is fictional, the underlying system faithfully simulates real-world trading mechanics:

- **Market Economics** — resource prices fluctuate randomly across planets, rewarding strategic buy-low-sell-high decisions.
- **Logistics Management** — fuel consumption and cargo capacity constrain travel and trade, mirroring real supply chain limitations.
- **Risk and Events** — random economic events force dynamic decision-making, reflecting real-world market volatility.
- **Persistent State** — game progress is saved and loaded via file I/O, simulating a stateful trading account.

The computing challenge lies in designing a well-structured, extensible system that cleanly models these real-world entities and interactions using the full spectrum of object-oriented programming principles.

## 2. OOP Principles Identification

---

This section identifies and justifies the OOP principles from the CT-260 syllabus that are applied throughout GalacticTrade.

### 2.1. Encapsulation

All class attributes are declared `private` or `protected`, and are accessible only through well-defined public methods. For example, the `Player` class hides `credits`, `fuel`, and `cargo` behind getter and setter methods, ensuring that no external code can directly corrupt the player's state.

```
1 class Player {  
2     private:
```

```

3     string name;
4     int credits;
5     int fuel;
6     int cargoSpace;
7     int currentCargo;
8 public:
9     int getCredits();
10    void deductCredits(int amount);
11    void addFuel(int amount);
12 };

```

Listing 1: Encapsulation in the Player class

## 2.2. Inheritance

The `Ship` class serves as an abstract base class. Three concrete ship types — `CargoShip`, `FighterShip`, and `SmugglerShip` — inherit from it, each with unique attributes and behaviour. Similarly, `Event` is an abstract base for `PirateAttack`, `MarketCrash`, and `ResourceBoom`. The `GameException` hierarchy extends `std::exception`.

```

1 class Ship { /* abstract base */ };
2
3 class CargoShip : public Ship {
4 public:
5     CargoShip() : Ship("Cargo Hauler", 100, 50, 3, 80) {}
6     string getShipType() override { return "Cargo Ship"; }
7     int getTravelCost() override { return 8; }
8 };

```

Listing 2: Inheritance — Ship subclasses

## 2.3. Polymorphism

Virtual functions allow runtime polymorphism across both the `Ship` and `Event` hierarchies. The same method call behaves differently depending on the actual object type at runtime.

```

1 // Base declares virtual functions
2 class Ship {
3 public:
4     virtual string getShipType() = 0;    // pure virtual
5     virtual int getTravelCost();         // virtual
6     virtual void showStats();            // virtual
7 };
8
9 // Called polymorphically      correct version runs at runtime

```

```

10 Ship* s = new FighterShip();
11 s->showStats();           // calls FighterShip::showStats()
12 s->getTravelCost();       // returns 12 (FighterShip's cost)

```

Listing 3: Polymorphism — virtual functions in Ship and Event

## 2.4. Abstraction

Both `Ship` and `Event` are abstract classes containing pure virtual functions (`= 0`). This prevents direct instantiation and enforces a contract that all derived classes must fulfill. Users of these classes interact only through the interface, not the implementation.

## 2.5. Static Members

The `GameEngine` class uses a `static` member to hold the single shared instance, a core requirement of the Singleton design pattern.

```

1 class GameEngine {
2 private:
3     static GameEngine* instance; // shared across all code
4     GameEngine();               // private constructor
5 public:
6     static GameEngine* getInstance();
7 };

```

Listing 4: Static member in GameEngine Singleton

## 2.6. Composition and Aggregation

`GameEngine` *owns* a `GameSave` object (composition) and manages `vector<Planet>` and `vector<Market>` containers (aggregation). The `Market` class uses `map<string, int>` to associate resource names with prices and stock levels.

## 2.7. Templates

A function template `getValidInput<T>()` is defined in `main.cpp` to safely read any numeric type from the console, handling invalid input gracefully without crashing.

```

1 template <typename T>
2 T getValidInput(string prompt) {
3     T value;
4     while (true) {
5         cout << prompt;
6         cin >> value;
7         if (cin.fail()) {
8             cin.clear();

```

```
9         cin.ignore(numeric_limits<streamsize>::max(), '\n')
10         ;
11         cout << "Invalid input! Please try again.\n";
12     } else { return value; }
13 }
```

Listing 5: Function template for input validation

## 3. System Design & UML

---

### 3.1. UML Class Diagram

Figure 1 presents the complete UML class diagram for GalacticTrade, showing all classes, their attributes, methods, visibility modifiers, and relationships.

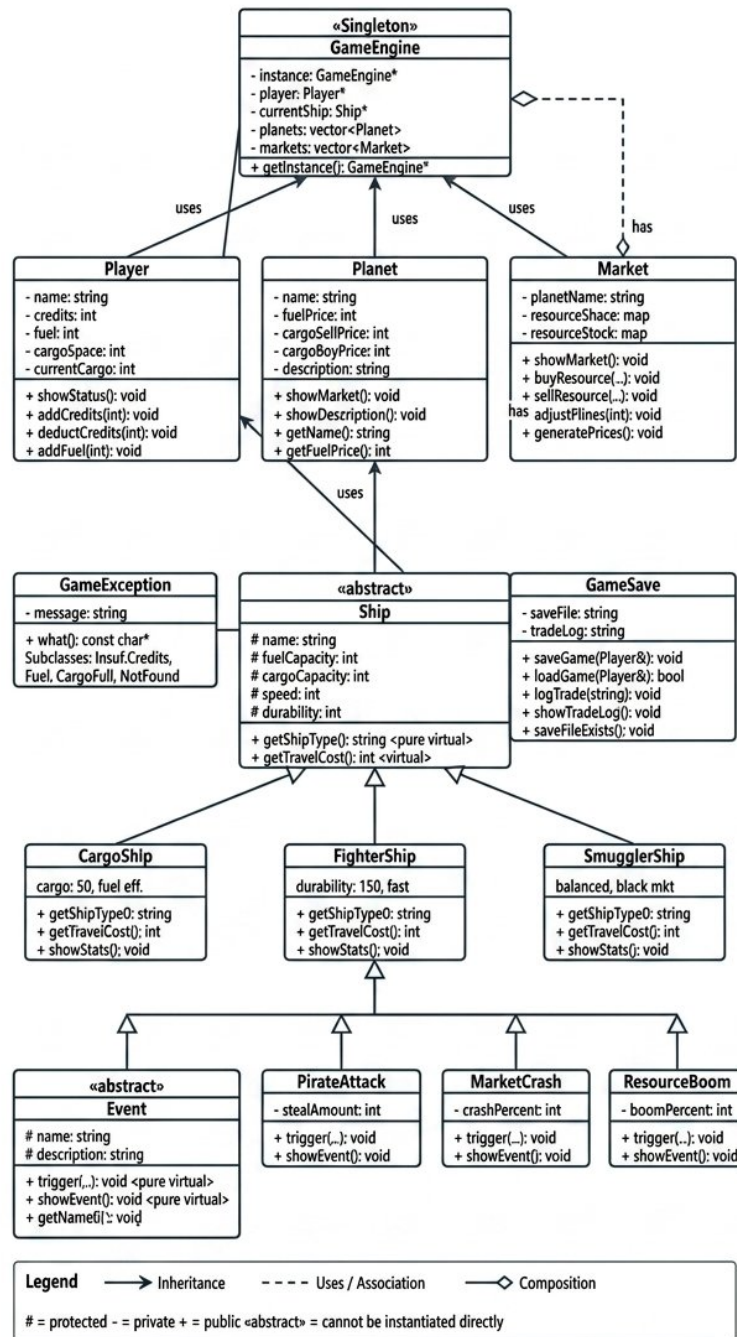


Figure 1: UML Class Diagram — GalacticTrade

### 3.2. Class Responsibilities

- **GameEngine** — Central controller (Singleton). Manages the game loop, planet/market initialisation, ship selection, travel, and event triggering.
- **Player** — Stores and manages the player’s credits, fuel, and cargo. Exposes safe methods for all state changes.
- **Ship** — Abstract base for all ship types. Defines the contract for travel cost and stats display via virtual functions.

- **CargoShip / FighterShip / SmugglerShip** — Concrete ship types, each with unique stats and travel cost overrides.
- **Planet** — Stores planet data including name, description, and pricing information.
- **Market** — Manages resource prices and stock using STL `map`. Handles buy, sell, and price adjustment operations.
- **Event** — Abstract base for random events. Subclasses `PirateAttack`, `MarketCrash`, and `ResourceBoom` each override `trigger()` differently.
- **GameSave** — Handles all file I/O: saving and loading player progress, and maintaining a trade log.
- **GameException** — Custom exception hierarchy extending `std::exception`, with specialised subclasses for each error type.

### 3.3. Key Relationships

- **Inheritance:** `CargoShip`, `FighterShip`, `SmugglerShip`  $\leftarrow$  `Ship`; `PirateAttack`, `MarketCrash`, `ResourceBoom`  $\leftarrow$  `Event`.
- **Composition:** `GameEngine` owns a `GameSave` instance.
- **Association:** `GameEngine` uses `Player`, `Planet`, `Market`, `Ship`, and `Event` objects.

## 4. Solution Definition

The following pseudocode describes the complete OOP-based solution for GalacticTrade. It covers all classes, their responsibilities, and the main program flow.

```
//
// CLASS DEFINITIONS
//

CLASS GameException extends std::exception:
    message : string
    what() → return message

SUBCLASSES:
    InsufficientCreditsException(required, available)
    InsufficientFuelException(required, available)
    CargoFullException(availableSpace)
    ResourceNotFoundException(resourceName)
    InvalidInputException()
```



```
//
```

```
CLASS Player:
```

```
    PRIVATE: name, credits=1000, fuel=50,  
             cargoSpace=10, currentCargo=0
```

```
    CONSTRUCTOR(name): set all starting values
```

```
    METHODS:
```

```
        getCredits(), getFuel(), getCargoSpace(),  
        getCurrentCargo()  
        addCredits(amount)    → credits += amount  
        deductCredits(amount) → credits -= amount  
        addFuel(amount)       → fuel += amount  
        consumeFuel(amount)   → fuel -= amount  
        addCargo(units)       → currentCargo += units  
        removeCargo(units)    → currentCargo -= units  
        showStatus()          → print name, credits,  
                               fuel, cargo
```

```
//
```

```
ABSTRACT CLASS Ship:
```

```
    PROTECTED: name, fuelCapacity, cargoCapacity,  
               speed, durability
```

```
    METHODS:
```

```
        virtual showStats()      → print all ship info  
        PURE VIRTUAL getShipType() → string  
        virtual getTravelCost()   → return 10
```

```
SUBCLASS CargoShip:
```

```
    fuel=100, cargo=50, speed=3, durability=80  
    getShipType() → "Cargo Ship"  
    getTravelCost() → 8
```

```
SUBCLASS FighterShip:
```

```
    fuel=80, cargo=20, speed=8, durability=150  
    getShipType() → "Fighter Ship"  
    getTravelCost() → 12
```

```
SUBCLASS SmugglerShip:
    fuel=90, cargo=30, speed=6, durability=100
    getShipType() → "Smuggler Ship"
    getTravelCost() → 10

//

CLASS Planet:
    PRIVATE: name, fuelPrice, cargoSellPrice,
              cargoBuyPrice, description
    CONSTRUCTOR() / CONSTRUCTOR(n, fp, sp, bp, desc)
    METHODS: getName(), showMarket(), showDescription()

//

CLASS Market:
    PRIVATE:
        planetName      : string
        resourcePrices   : map<string, int>
        resourceStock    : map<string, int>

    generatePrices():
        Iron      → price: 20-50,  stock: 10-30
        Spice     → price: 50-100, stock: 5-15
        Fuel Cell → price: 30-50,  stock: 8-20
        Gems      → price: 80-150, stock: 3-11
        Food      → price: 10-25,  stock: 15-35

    buyResource(player, resource, units):
        IF resource not in map      → THROW ResourceNotFoundException
        IF units > stock            → THROW GameException
        IF credits < totalCost      → THROW InsufficientCreditsException
        IF cargo + units > cargoSpace → THROW CargoFullException
        player.deductCredits(cost)
        player.addCargo(units)
        stock[resource] -= units

    sellResource(player, resource, units):
        IF resource not in map → THROW ResourceNotFoundException
        IF cargo < units      → THROW GameException
```

```

        player.addCredits(earned)
        player.removeCargo(units)
        stock[resource] += units

//

ABSTRACT CLASS Event:
    PURE VIRTUAL trigger(player, market)
    PURE VIRTUAL showEvent()

    SUBCLASS PirateAttack:
        stealAmount = random(50-199)
        trigger(): deduct credits from player

    SUBCLASS MarketCrash:
        crashPercent = random(20-49)
        trigger(): market.adjustPrices(-crashPercent)

    SUBCLASS ResourceBoom:
        boomPercent = random(20-59)
        trigger(): market.adjustPrices(+boomPercent)

//

CLASS GameSave:
    saveGame(player)    → WRITE to savegame.txt
    loadGame(player)    → READ from savegame.txt
    logTrade(details)   → APPEND to tradelog.txt
    showTradeLog()      → PRINT tradelog.txt
    saveFileExists()    → RETURN true if file opens

//

CLASS GameEngine <<SINGLETON>>:
    PRIVATE: instance=nullptr, player, currentShip,
             planets: vector<Planet>,
             markets: vector<Market>

    getInstance():
        IF instance == nullptr → instance = new GameEngine()
        RETURN instance

```

```
startGame():
    IF saveFile exists → ASK to load
        IF yes → player = new Player("temp")
            loadGame(player)
        ELSE → READ name; player = new Player(name)
    handleShipSelection() // always runs

travel():
    cost = currentShip.getTravelCost() // POLYMORPHIC
    IF fuel < cost → THROW InsufficientFuelException
    player.consumeFuel(cost)
    currentPlanetIndex = destination
    triggerRandomEvent()

triggerRandomEvent():
    IF random < 30%:
        event = new PirateAttack/MarketCrash/ResourceBoom
        event.trigger(player, market) // POLYMORPHIC
        DELETE event

handleMarket():
    READ choice (1=Buy, 2=Sell, 3=Back)
    IF buy → market.buyResource(...)
        gameSave.logTrade(...)
    IF sell → market.sellResource(...)
        gameSave.logTrade(...)

//
// MAIN PROGRAM
//

FUNCTION getValidInput<T>(): // TEMPLATE
    LOOP:
        READ input
        IF invalid → clear, ask again
        ELSE → RETURN value

MAIN():
    engine = GameEngine.getInstance() // Singleton
    engine.startGame()
```

```

WHILE running:
    SHOW menu
    choice = getValidInput<int>()
    TRY:
        SWITCH choice:
            1 → showCurrentPlanet()
            2 → handleMarket()
            3 → showShipStats()
            4 → travel()
            5 → showPlayerStatus()
            6 → showTradeLog()
            7 → saveGame()
            8 → saveAndQuit(); running = false
        DEFAULT → THROW InvalidInputException
    CATCH InsufficientFuelException → print error
    CATCH InvalidInputException    → print error
    CATCH GameException            → print error
    CATCH (...)                    → print error

```

## 5. Design Pattern Application

---

### 5.1. Pattern Used: Singleton

GalacticTrade implements the **Singleton** design pattern in the `GameEngine` class. The Singleton pattern ensures that a class has *exactly one instance* throughout the entire lifetime of the program, and provides a single global access point to that instance.

### 5.2. Why Singleton for GameEngine?

The `GameEngine` is the central controller of the entire application. It owns the player, the current ship, all planets, all markets, and the save system. Having more than one `GameEngine` running simultaneously would create conflicting game states, duplicate data, and unpredictable behaviour. The Singleton pattern enforces this constraint at the language level.

### 5.3. Implementation

```

1 class GameEngine {
2 private:
3     static GameEngine* instance; // one instance only
4     GameEngine();                // private blocks new
5     GameEngine()

```

```
6 public:
7     // Only way to access the engine
8     static GameEngine* getInstance() {
9         if (instance == nullptr)
10             instance = new GameEngine();
11         return instance;
12     }
13     // Copying disabled entirely
14     GameEngine(const GameEngine&) = delete;
15     GameEngine& operator=(const GameEngine&) = delete;
16 };
```

Listing 6: Singleton pattern in GameEngine

## 5.4. How It Improves the Design

- **Controlled access** — no code anywhere in the project can accidentally create a second engine.
- **Global consistency** — all parts of the program always refer to the same game state.
- **Lazy initialisation** — the engine is only created when first needed, not at program startup.
- **Copy prevention** — the deleted copy constructor and assignment operator make accidental duplication a compile-time error.

## 5.5. Usage in main.cpp

```
1 int main() {
2     GameEngine* engine = GameEngine::getInstance();
3     engine->startGame();
4     // game loop ...
5 }
```

Listing 7: Singleton accessed in main()

# 6. Limitations & Future Work

## 6.1. Current Limitations

1. **No graphical user interface** — GalacticTrade runs entirely in the console. While functional, the text-based interface limits the user experience compared to a GUI-based application.

2. **Limited planet and resource variety** — The simulation includes only 5 planets and 5 resource types. This restricts the depth of trading strategy available to the player.
3. **No multiplayer support** — The system is designed for a single player only. There is no mechanism for multiple traders to interact, compete, or trade with each other.
4. **Basic save system** — The save mechanism stores only the player's core stats. Ship type, current planet, and market state are not persisted, meaning some progress is lost on reload.
5. **No ship upgrade system** — While three ship types exist, players cannot upgrade or modify their ship mid-game. A dynamic upgrade system would add strategic depth.
6. **Random events lack player agency** — Events such as pirate attacks trigger automatically with no option for the player to respond, evade, or negotiate, limiting interactivity.

## 6.2. Potential Future Enhancements

- Implement a graphical interface using a library such as SFML.
- Expand the universe with more planets, resources, and dynamic trade routes.
- Add a ship upgrade system with purchasable components.
- Introduce a full event-response system giving players choices during events.
- Implement a complete save system that persists the entire game state.

## 7. Team Contribution Table

---

Table 1 outlines the individual contributions of each team member to the GalacticTrade project.

Table 1: Team Member Contributions

| Name                 | Roll No.             | Contribution                                                                                                                                                                                                                                                                                                                      |
|----------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Amna Kaleem</b>   | <b>Ali</b> CT-25264  | <p><code>Player.h / .cpp</code> — Player class with encapsulation</p> <p><code>Planet.h / .cpp</code> — Planet data and market display</p> <p><code>Ship.h / .cpp</code> — Abstract Ship base class and all three subclasses (CargoShip, FighterShip, SmugglerShip) with inheritance and polymorphism</p>                         |
| <b>Zainab Nadeem</b> | CT-25265             | <p><code>Event.h / .cpp</code> — Abstract Event base class and subclasses (PirateAttack, MarketCrash, ResourceBoom)</p> <p><code>GameSave.h / .cpp</code> — File I/O system for saving, loading, and trade logging</p>                                                                                                            |
| <b>Faarid Shaikh</b> | <b>Amir</b> CT-25276 | <p><code>Exceptions.h</code> — Custom exception hierarchy</p> <p><code>Market.h / .cpp</code> — STL map-based market with buy/sell logic</p> <p><code>GameEngine.h / .cpp</code> — Singleton game controller and main game loop</p> <p><code>main.cpp</code> — Entry point, menu, template input function, exception handling</p> |



## References

---

1. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
2. Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). *C++ Primer* (5th ed.). Addison-Wesley.
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
4. cppreference.com. (2024). *C++ Standard Library Reference*. Retrieved from <https://en.cppreference.com>